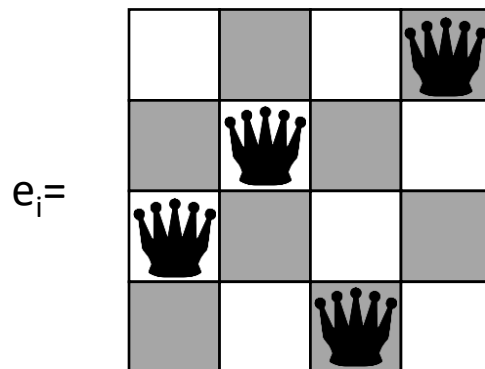


Objectiu de la classe: Analitzar i aprofundir
ens els algorismes de cerca local

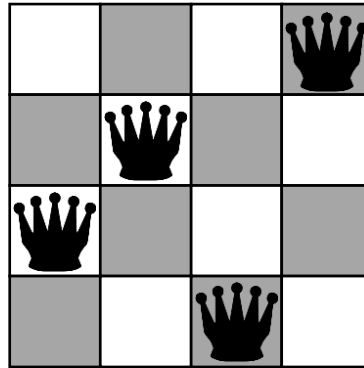
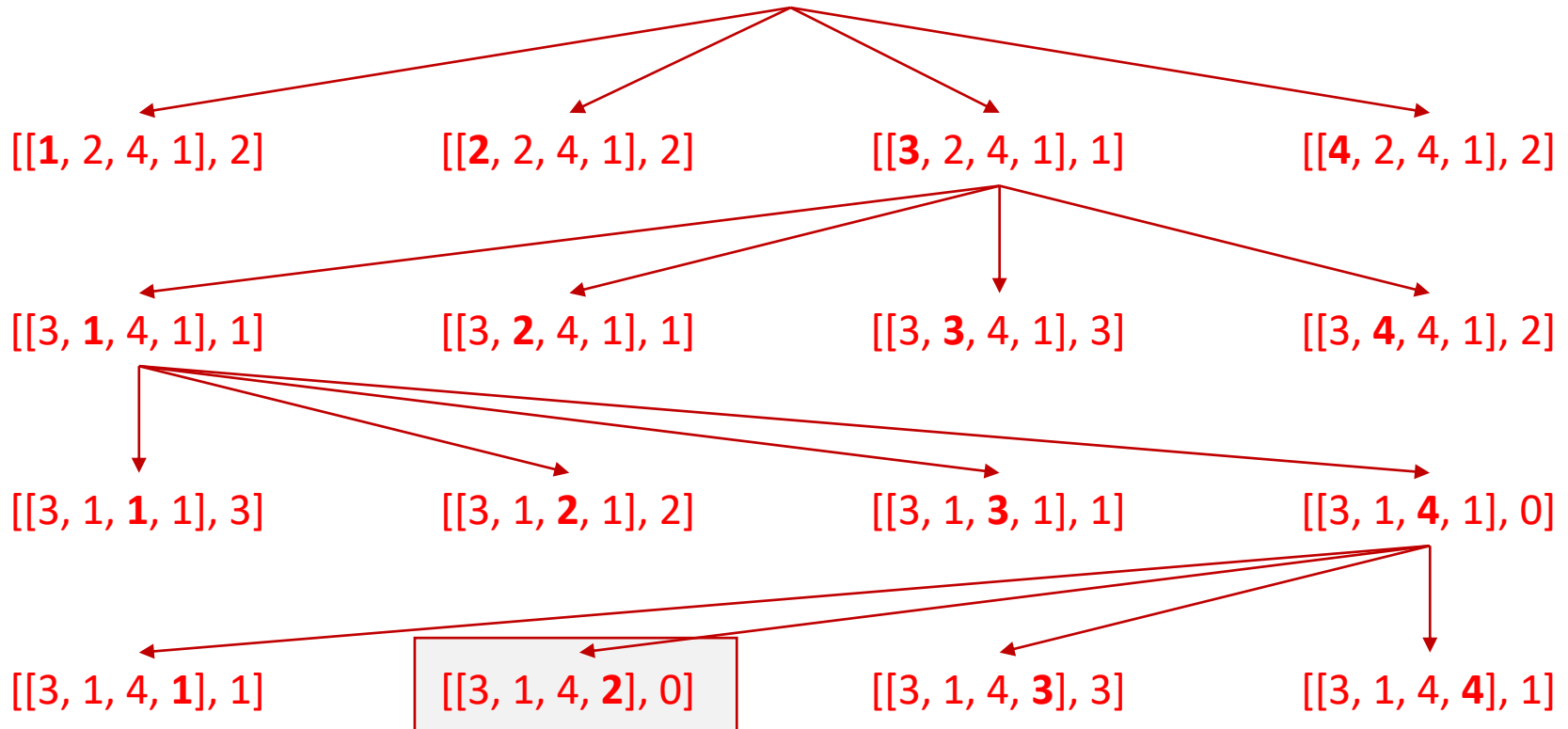
Exercici 1. Suposem que volem solucionar el problema de les 4 reines fent servir un algorisme de Cerca Local. Representarem el taulell com una llista que conté 4 elements: el 1^r element conté la fila on es troba la reina de la columna 1, el 2ⁿ element conté la fila on es troba la reina de la columna 2, i així successivament fins al 4^{rt} element.

L'estat final és qualsevol estat en que les 4 reines no es maten entre elles. En cada nivell expandim la reina d'una columna concreta a cada una de les 4 files, començant per la columna 1.

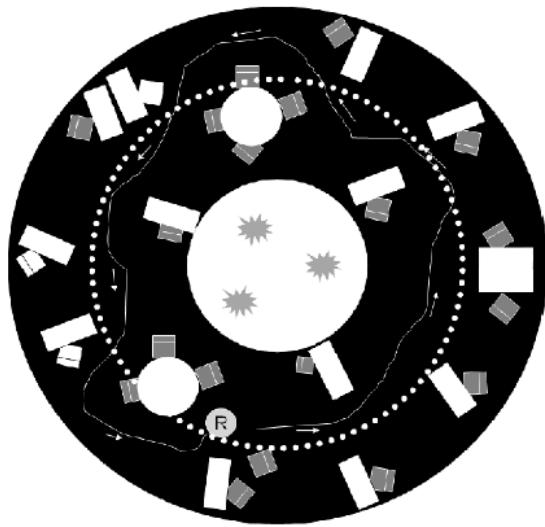
Completa l'arbre que construirà l'algorisme "*hill-climbing*" si partim de l'estat e_i . I fem servir l'heurística vista a classe, on per a la reina de la columna que analitzem, comptem quantes reines pot matar.



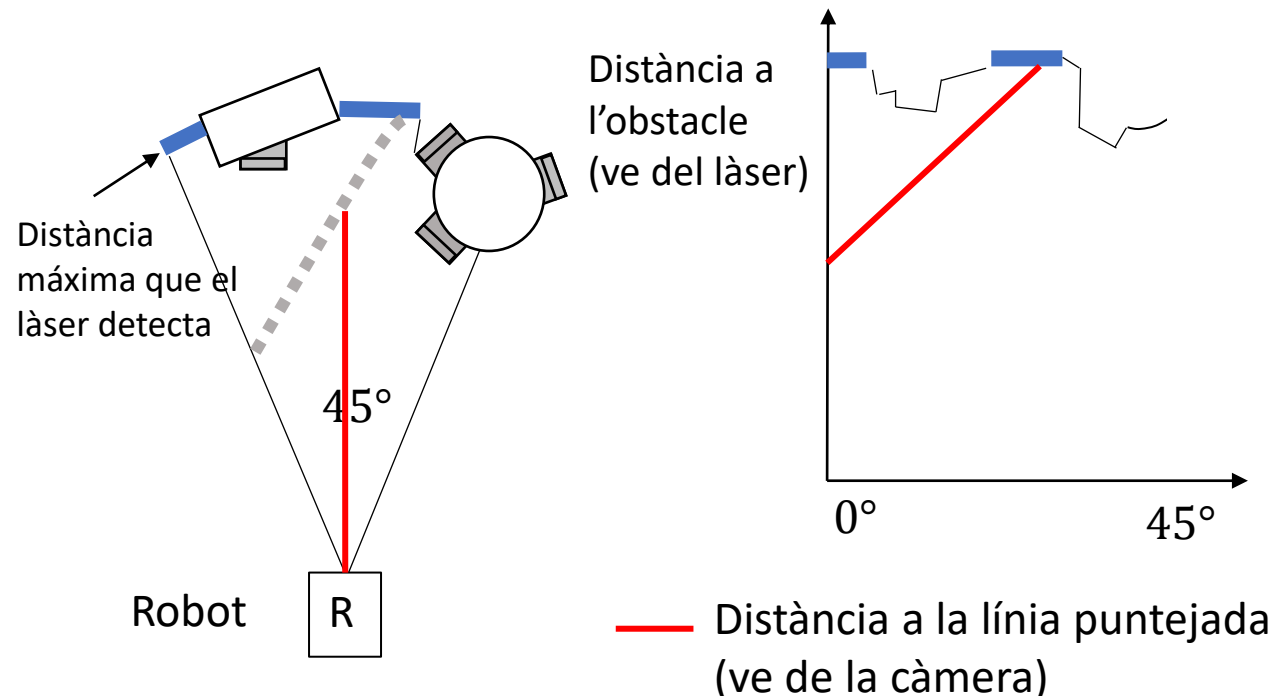
$e_i = [3, 2, 4, 1]$

$e_i =$  $[[3, 2, 4, 1], N/A]$ 

Exercici 2: Supposeu que heu de fer un agent que controla un robot que navega per un edifici circular. El robot porta cafè calent a sobre per a que la gent que treballa a l'edifici es pugui servir quan passa el robot. Per tant, el robot s'ha de limitar a circular per l'edifici mentre ningú el pari, i ha d'anar esquivant tots els obstacles que li apareixen pel davant. Per a poder navegar en condicions el robot porta un sensor làser que detecta a quina distància es troben tots els objectes que té el robot al davant en cada moment dins d'un angle de 45 graus. A més a més porta una càmera que detecta una línia blanca puntejada dibuixada al terra, que passa pel centre de l'edifici i que determina la trajectòria ideal del robot, és a dir per on hauria de passar si no hi hagués cap obstacle.



Mapa de l'edifici



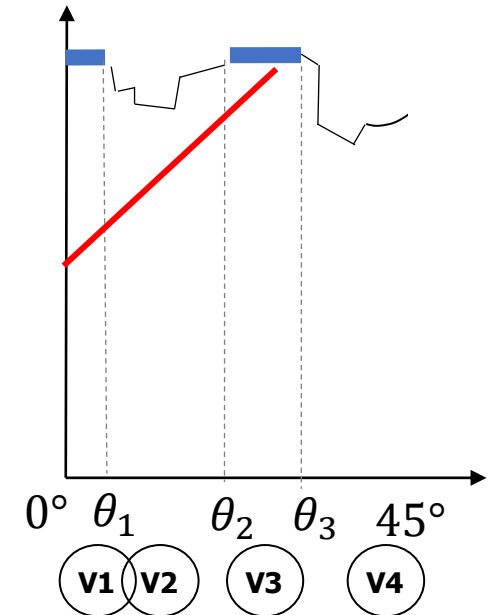
Contesteu a les següents preguntes:

(a) Es pot solucionar aquest problema com una cerca?, de quin tipus?

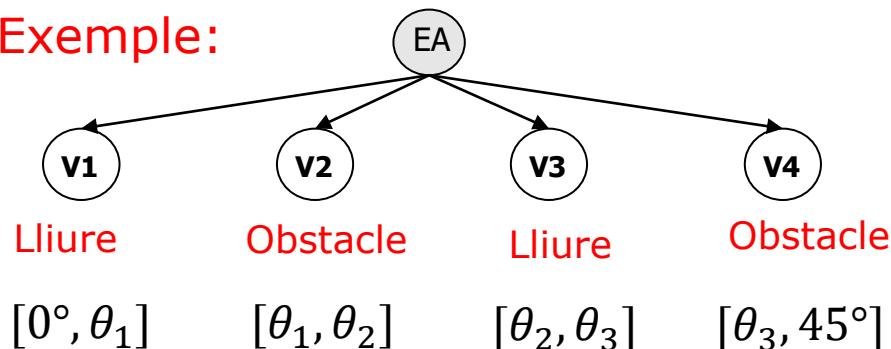
Sí, com una cerca local, perquè l'entorn és variable i no es coneix un estat objectiu més enllà d'intentar optimitzar el fet de mantenir-se sobre la línia puntejada sense topiar amb cap obstacle.

(b) Què serien els nodes i què serien les branques?

Un node representa el conjunt d'angles que agrupen una regió que és obstacle o està lliure.



Exemple:



$$d_{obstacle}(V_i) = \frac{\sum_{\theta \in V_i} d_{obstacle}(\theta)}{\#(\theta \in V_i)}$$

$$d_{línia}(V_i) = \frac{\sum_{\theta \in V_i} d_{línia}(\theta)}{\#(\theta \in V_i)}$$

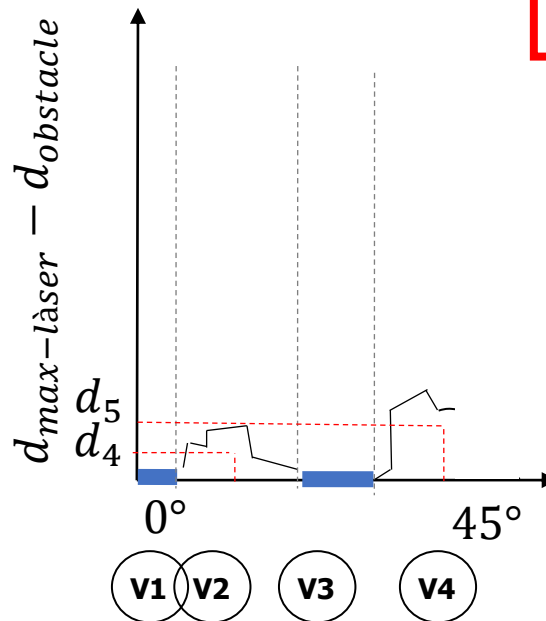
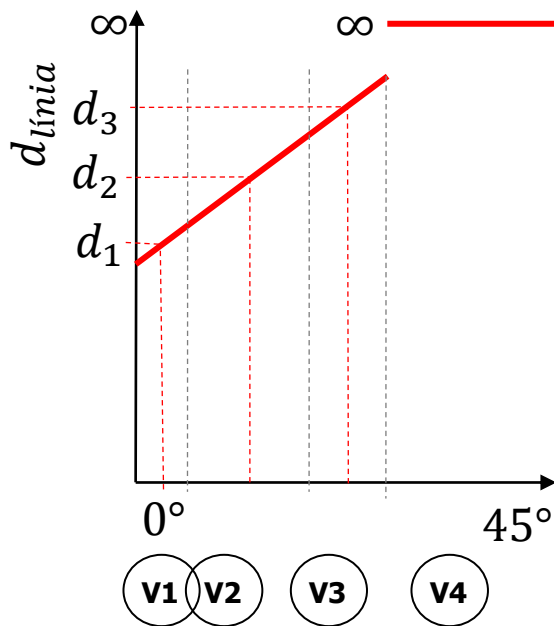
c) Quin algorisme faries servir? **Steepest ascent (descent)**
(no coneixem l'estat objectiu, no guardem el camí)

d) Quina funció heurística es podria definir:

$$h_1(V_i) = d_{línia}(V_i) + (d_{max-làser} - d_{obstacle}(V_i)) \quad (\text{descent})$$

$$h_2(V_i) = -d_{línia}(V_i) + d_{obstacle}(V_i) \quad (\text{ascent})$$

Considerem distància
 infinita en els estats on no
 es veu la línia puntejada



$$h_1(V1) = d_1 + 0$$

Estat
 Següent

$$h_1(V2) = d_2 + d_4$$

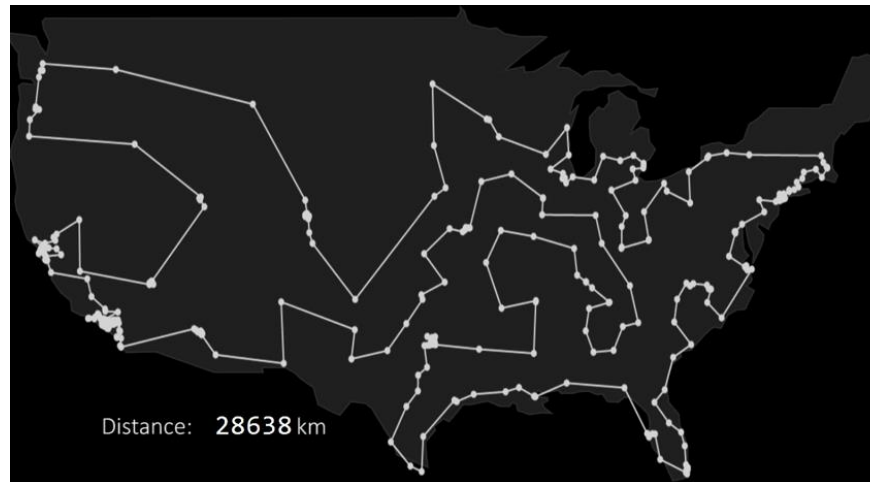
$$h_1(V3) = d_3 + 0$$

$$h_1(V4) = \infty + d_5 = \infty$$

Exercici 3: el viatjant de comerç

<https://www.youtube.com/watch?v=SC5CX8drAtU>

Donat un conjunt de ciutats, de les quals coneixem la seva localització, sabries determinar quina és la ruta més curta que ha de fer un comerciant per visitar totes les ciutats i retornar a la ciutat origen?

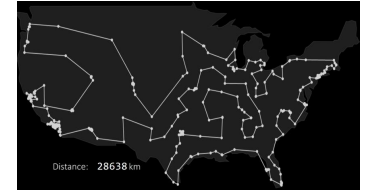


a) Es pot solucionar aquest problema com una cerca?, de quin tipus?

Sí, com una cerca local, perquè l'estat objectiu (seqüència de ciutats que s'han de visitar) no és conegut. Només coneixem la mètrica que ens permet establir si un estat és millor que un altre.

Exercici 3: el viatjant de comerç

<https://www.youtube.com/watch?v=SC5CX8drAtU>



Donat un conjunt de ciutats, de les que coneixem la localització, quina és la ruta més curta que visita totes les ciutats i retorna a la ciutat origen?

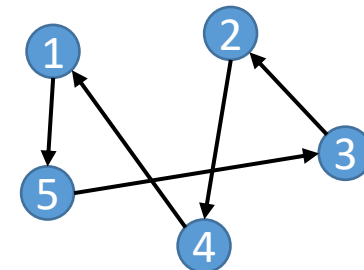
b) Quin podria ser un estat inicial?

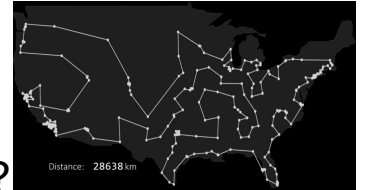
Un estat, i per tant un node en l'arbre de cerca, serà una llista de ciutats que comença per la ciutat origen i acaba en la ciutat, en la que totes les ciutats hi son presents. Aquesta llista indica l'ordre de visita de cada una de les ciutats. Suposem que hi ha camí entre qualsevol parella de ciutats al que hi tenim associat el cost en distància. En el cas que no hi hagués camí el cost seria infinit. Un camí inicial seria qualsevol permutació del les ciutats 2 fins a n, si suposem que comencem a la ciutat 1. Exemple per a 5 ciutats:

[1, 5, 3, 2, 4, 1], **196 km**

matriu de distància

	1	2	3	4	5
1	0	30	58	47	25
2	30	0	27	37	40
3	58	27	0	31	60
4	47	37	31	0	32
5	25	40	60	32	0



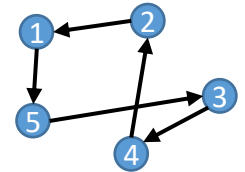
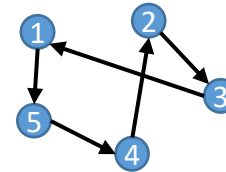
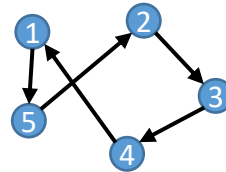
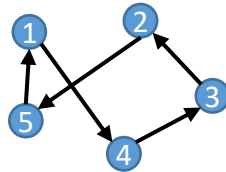
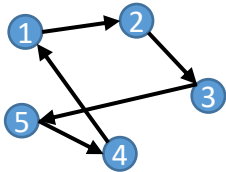
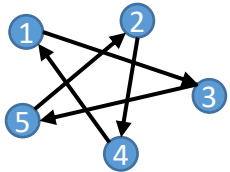
Exercici 3: el viatjant de comerç

c) Donat un estat, com definiries els estats següents possibles?

Els estats als que podem anar a partir d'un estat donat vindran donats per totes aquelles variacions mínimes que puguem fer al itinerari. Aquest canvi podria ser intercanviar l'ordre de qualsevol parella de ciutats (excepte l'origen). En l'exemple anterior

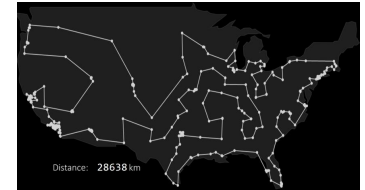


[1,3,5,2,4,1], 242 [1,2,3,5,4,1], 196 [1,4,3,2,5,1], 170 [1,5,2,3,4,1], 170 [1,5,4,2,3,1], 179 [1,5,3,4,2,1], 183



Exercici 3: el viatjant de comerç

<https://www.youtube.com/watch?v=SC5CX8drAtU>



Donat un conjunt de ciutats, de les que coneixem la localització, quina és la ruta més curta que visita totes les ciutats i retorna a la ciutat origen?

d) Quina heurística usaries per a establir si s'adopta un determinat canvi o no? Com ho optimitzaries?

La suma de les distàncies de cada ciutat a la ciutat anterior de la llista. Com que només s'intercanvien dues ciutats, podem mantenir el cost del estat pare, modificant les distàncies dels trajectes que canvien.

e) Simulated Annealing seria millor opció que Steepest Ascent / Decent?

- Permet sortir de mínims locals sense haver de reiniciar la cerca un altra vegada amb diferents condicions inicials.
- És més eficient. El Simulated Annealing, en cada intent, només fem una avaluació. Per exemple, en un mapa de 300 ciutats, tindríem 44551 avaluacions amb cerca local per cada pas.
- La "temperatura" (que va baixant en cada iteració) ens indica en cada step fins a quin punt ens podem "equivocar", i seleccionar un altre step encara que empitjori l'heurística. Es decideix de manera probabilística.